

SQL SUBQUERIES

What is a Subquery in SQL?

A subquery is a SQL query written inside another SQL query. The inner query runs first, produces a result, and that result is then used by the outer query to complete its work.

Think of it like this:

- The INNER query is the helper. It fetches some data.
- The OUTER query uses that data to answer your main question.

A subquery is always enclosed in parentheses ().

Basic Structure

```
SELECT column_name  
FROM table_name  
WHERE column_name OPERATOR (SELECT column_name FROM table_name WHERE  
condition);  
| _____ This is the SUBQUERY _____ |
```

Key Rule: The subquery always runs FIRST. Its result is passed to the outer query.

Subqueries can appear in:

- WHERE clause (most common)
- FROM clause (derived tables)
- SELECT clause (scalar subqueries)
- HAVING clause

Step 2: Why is a Subquery Required?

Many real-world questions cannot be answered with a single simple SELECT. You first need to calculate or look up something, and then use that result to filter or transform data.

Real-World Scenarios Where Subqueries Are Needed

Scenario	What You Need
Find employees earning above average salary	First calculate average salary, then compare each employee's salary with that average.
Find departments with at least one employee earning > 80000	First find qualifying employee records, then match departments.
Find products not ordered by any customer	First get all ordered product IDs, then find products NOT in that list.
Find the second highest salary	First find the highest salary, then find the highest salary below that value.

Bottom Line: Subqueries let you answer questions that depend on a dynamic computed value, not a fixed value you already know.

Step 3: Subquery with a Simple Story Example

Story: Riya is the HR manager of a company. She wants to find all employees who earn MORE than the average salary of the company. She does not know the average salary in advance. So she first has to calculate the average, then use it to find the employees.

The Tables

Table: **EMPLOYEES**

EMP_ID	NAME	DEPARTMENT	SALARY	MANAGER_ID
101	Alice	HR	50000	NULL
102	Bob	IT	80000	101
103	Charlie	IT	75000	101
104	Diana	Finance	90000	NULL
105	Eve	HR	48000	101
106	Frank	Finance	95000	104
107	Grace	IT	82000	101

```
-- Create Table
CREATE TABLE Employees (
  EMP_ID INT PRIMARY KEY,
  NAME VARCHAR(50),
  DEPARTMENT VARCHAR(50),
  SALARY INT,
  MANAGER_ID INT
```

);

-- Insert Data

```
INSERT INTO Employees (EMP_ID, NAME, DEPARTMENT, SALARY, MANAGER_ID) VALUES
(101, 'Alice', 'HR', 50000, NULL),
(102, 'Bob', 'IT', 80000, 101),
(103, 'Charlie', 'IT', 75000, 101),
(104, 'Diana', 'Finance', 90000, NULL),
(105, 'Eve', 'HR', 48000, 101),
(106, 'Frank', 'Finance', 95000, 104),
(107, 'Grace', 'IT', 82000, 101);
```

Riya's Thinking Process

Step A: First, find the average salary of all employees.

```
SELECT AVG(SALARY) FROM EMPLOYEES;
```

Output of Step A:

AVG(SALARY)
74285.71

Step B: Now find all employees whose salary is greater than 74285.71.

```
SELECT NAME, SALARY FROM EMPLOYEES WHERE SALARY > 74285.71;
```

The Problem: The value 74285.71 is hardcoded. If new employees join, the average changes. The query becomes wrong.

Solution: Use a Subquery! Let SQL calculate the average automatically inside the query.

The Subquery (One Single Query that Does Both Steps)

```
SELECT NAME, SALARY
FROM EMPLOYEES
WHERE SALARY > (SELECT AVG(SALARY) FROM EMPLOYEES);
| _____ SUBQUERY _____ |
```

Expected Output:

NAME	SALARY
Bob	80000
Diana	90000
Frank	95000

NAME	SALARY
Grace	82000

How to Read This Query

1. SQL runs the INNER query first: `SELECT AVG(SALARY) FROM EMPLOYEES` => result is 74285.71
2. SQL substitutes that result: `WHERE SALARY > 74285.71`
3. Outer query filters employees and returns names and salaries.



Step 4: Types of Subqueries in SQL

Type	Description	Returns
Single Row Subquery	Returns exactly one row and one column.	One single value (e.g., one salary)
Multi Row Subquery	Returns more than one row but one column.	A list of values (e.g., list of dept names)
Multi Column Subquery	Returns more than one column.	Multiple columns per row
Correlated Subquery	References the outer query's column inside inner query. Runs once per outer row.	Depends on outer query
Nested Subquery	A subquery inside another subquery.	Any of the above

Quick Memory Tip: Single Row => use =, >, < | Multi Row => use IN, ANY, ALL | Correlated => uses outer table alias inside inner query

Step 5: Single Row Subquery

A Single Row Subquery returns exactly ONE value (one row, one column). You use this with comparison operators like =, >, <, >=, <=, <>.

Table Used: EMPLOYEES

EMP_ID	NAME	DEPARTMENT	SALARY	MANAGER_ID
101	Alice	HR	50000	NULL
102	Bob	IT	80000	101
103	Charlie	IT	75000	101
104	Diana	Finance	90000	NULL
105	Eve	HR	48000	101
106	Frank	Finance	95000	104
107	Grace	IT	82000	101

Operators Used with Single Row Subquery

Operator	Meaning
=	Equal to the subquery result
>	Greater than the subquery result

Operator	Meaning
<	Less than the subquery result
>=	Greater than or equal to
<=	Less than or equal to
<>	Not equal to

Example 1: Find the employee with the highest salary

```
SELECT NAME, SALARY
FROM EMPLOYEES
WHERE SALARY = (SELECT MAX(SALARY) FROM EMPLOYEES);
```

Step-by-step:

4. Inner query: SELECT MAX(SALARY) FROM EMPLOYEES => 95000
5. Outer query becomes: WHERE SALARY = 95000
6. SQL fetches the employee with salary 95000

Expected Output:

NAME	SALARY
Frank	95000

Example 2: Find employees who earn more than Alice (EMP_ID = 101)

```
SELECT NAME, SALARY
FROM EMPLOYEES
WHERE SALARY > (SELECT SALARY FROM EMPLOYEES WHERE EMP_ID = 101);
```

Step-by-step:

7. Inner query: SELECT SALARY FROM EMPLOYEES WHERE EMP_ID = 101 => 50000
8. Outer query becomes: WHERE SALARY > 50000
9. SQL returns all employees with salary greater than 50000

Expected Output:

NAME	SALARY
Bob	80000
Charlie	75000
Diana	90000
Frank	95000
Grace	82000

Example 3: Find the department of the lowest-paid employee

```
SELECT NAME, DEPARTMENT, SALARY
FROM EMPLOYEES
WHERE SALARY = (SELECT MIN(SALARY) FROM EMPLOYEES);
```

Step-by-step:

10. Inner query: SELECT MIN(SALARY) FROM EMPLOYEES => 48000
11. Outer query: WHERE SALARY = 48000
12. Returns the employee with salary 48000

Expected Output:

NAME	DEPARTMENT	SALARY
Eve	HR	48000

Important Error: If your subquery returns more than one row and you use =, SQL will throw an error: 'single-row subquery returns more than one row'. Always make sure the inner query returns exactly one value for single row subqueries.

Step 6: Multi Row Subquery

A Multi Row Subquery returns MORE THAN ONE ROW. Since the result is a list, you cannot use = directly. You must use special operators: IN, ANY, or ALL.

Rule: Single Row => use = | Multi Row => use IN, ANY, ALL

Table Used: EMPLOYEES

EMP_ID	NAME	DEPARTMENT	SALARY	MANAGER_ID
101	Alice	HR	50000	NULL
102	Bob	IT	80000	101
103	Charlie	IT	75000	101
104	Diana	Finance	90000	NULL
105	Eve	HR	48000	101
106	Frank	Finance	95000	104
107	Grace	IT	82000	101

Example 1: Find all employees who work in the IT or Finance department (using IN)

We want to find employees whose department is in the list of departments that have average salary above 70000.

```
SELECT NAME, DEPARTMENT
FROM EMPLOYEES
WHERE DEPARTMENT IN (SELECT DEPARTMENT FROM EMPLOYEES
                      GROUP BY DEPARTMENT
                      HAVING AVG(SALARY) > 70000) ;
```

Step-by-step:

13. Inner query: Groups employees by department and returns departments with avg salary > 70000.
14. Result of inner query: IT, Finance
15. Outer query: WHERE DEPARTMENT IN ('IT', 'Finance')
16. Returns all employees in those departments

Expected Output:

NAME	DEPARTMENT
Bob	IT
Charlie	IT
Diana	Finance
Frank	Finance
Grace	IT

Example 2: Find employees who earn more than at least one HR employee (using ANY)

```
SELECT NAME, SALARY
FROM EMPLOYEES
WHERE SALARY > ANY (SELECT SALARY FROM EMPLOYEES WHERE DEPARTMENT =
                    'HR') ;
```

Step-by-step:

17. Inner query returns HR salaries: 50000, 48000
18. > ANY means: salary > at least one value in the list (i.e., salary > 48000)
19. Employees earning more than 48000 are returned

Expected Output:

NAME	SALARY
Alice	50000
Bob	80000
Charlie	75000
Diana	90000
Frank	95000
Grace	82000

Example 3: Find employees who earn more than ALL Finance employees


```
SELECT NAME, SALARY
FROM EMPLOYEES
WHERE SALARY > ALL (SELECT SALARY FROM EMPLOYEES WHERE DEPARTMENT =
'Finance');
```

Step-by-step:

- 20. Inner query returns Finance salaries: 90000, 95000
- 21. > ALL means: salary must be greater than EVERY value in the list (i.e., salary > 95000)
- 22. No employee earns more than 95000, so no rows are returned

Expected Output:

NAME	SALARY
(No rows returned)	

Step 7: Operators IN, ANY, and ALL Explained

Operator	Meaning	Example in Plain English
IN	TRUE if value matches any value in the subquery list	Salary is in the list (50000, 80000, 90000)
ANY	TRUE if comparison is true for AT LEAST ONE value in the list	Salary > any one value in the list
ALL	TRUE only if comparison is true for EVERY value in the list	Salary > every value in the list

IN Operator: Example 1

Find employees who work in departments located in Bangalore or Delhi.

```
SELECT NAME, DEPARTMENT
FROM EMPLOYEES
WHERE DEPARTMENT IN (SELECT DEPT_NAME FROM DEPARTMENTS
WHERE LOCATION IN ('Bangalore', 'Delhi'));
```

DEPARTMENTS Table (for reference):

DEPT_ID	DEPT_NAME	LOCATION
10	HR	Mumbai
20	IT	Bangalore
30	Finance	Delhi

Inner query returns: IT, Finance (departments in Bangalore or Delhi)

Expected Output:

NAME	DEPARTMENT
Bob	IT
Charlie	IT
Diana	Finance
Frank	Finance
Grace	IT

IN Operator: Example 2

Find employees who are managers (i.e., their EMP_ID appears in the MANAGER_ID column).

```
SELECT NAME
FROM EMPLOYEES
WHERE EMP_ID IN (SELECT DISTINCT MANAGER_ID FROM EMPLOYEES
                 WHERE MANAGER_ID IS NOT NULL);
```

Inner query returns: 101, 104 (the manager IDs)

Expected Output:

NAME
Alice
Diana

ANY Operator: Example 1

Find employees who earn more than at least one person in the Finance department.

```
SELECT NAME, SALARY
FROM EMPLOYEES
WHERE SALARY > ANY (SELECT SALARY FROM EMPLOYEES
                   WHERE DEPARTMENT = 'Finance');
```

Inner query returns Finance salaries: 90000, 95000

> ANY means: salary must be greater than 90000 (the smallest value in the list)

Expected Output:

NAME	SALARY
Frank	95000

ANY Operator: Example 2

Find employees earning less than at least one IT employee.

```
SELECT NAME, SALARY
FROM EMPLOYEES
WHERE SALARY < ANY (SELECT SALARY FROM EMPLOYEES
                   WHERE DEPARTMENT = 'IT');
```

IT salaries: 80000, 75000, 82000. < ANY means salary < 82000 (the largest in list)

Expected Output:

NAME	SALARY
Alice	50000
Charlie	75000
Eve	48000
Bob	80000

ALL Operator: Example 1

Find employees who earn more than ALL HR employees.

```
SELECT NAME, SALARY
FROM EMPLOYEES
WHERE SALARY > ALL (SELECT SALARY FROM EMPLOYEES
                    WHERE DEPARTMENT = 'HR');
```

HR salaries: 50000, 48000. > ALL means salary > 50000 (must beat the maximum)

Expected Output:

NAME	SALARY
Bob	80000
Charlie	75000
Diana	90000
Frank	95000
Grace	82000

ALL Operator: Example 2

Find employees earning less than ALL Finance employees.

```
SELECT NAME, SALARY
FROM EMPLOYEES
WHERE SALARY < ALL (SELECT SALARY FROM EMPLOYEES
                    WHERE DEPARTMENT = 'Finance');
```

Finance salaries: 90000, 95000. < ALL means salary < 90000

Expected Output:

NAME	SALARY
Alice	50000
Bob	80000
Charlie	75000
Eve	48000
Grace	82000

Step 8: What is a Correlated Subquery?

In a regular subquery, the inner query runs ONCE and passes its result to the outer query. That result is fixed.

In a Correlated Subquery, the inner query references a column from the OUTER query. Because of this, the inner query runs ONCE FOR EACH ROW of the outer query.

Regular Subquery	Correlated Subquery
Inner query runs ONCE. Result is fixed and passed to outer query.	Inner query runs once FOR EACH ROW of the outer query. The result changes per row.
Inner query does NOT reference outer table.	Inner query REFERENCES a column from outer table (uses alias).

When Should You Use a Correlated Subquery?

Think of correlated subqueries when you want to compare each row of a table against a group-level result SPECIFIC TO THAT ROW's category.

Example questions that need correlated subqueries:

- Find employees who earn more than the average salary of their OWN department.
- Find students who scored higher than the average score of their OWN class.
- Find products priced higher than the average price in their OWN category.

Trigger Phrase: Whenever you see 'their own department', 'their own class', 'same group' — think Correlated Subquery.

Step 9: Example of a Correlated Subquery

Question: Find employees who earn more than the average salary of their own department.

Why regular subquery fails here: A regular subquery can only give you the overall company average. It cannot give you the average PER DEPARTMENT for each row.

How to Think Through This

23. For Alice (HR dept) => calculate average of HR => compare Alice's salary to that average
24. For Bob (IT dept) => calculate average of IT => compare Bob's salary to that average
25. For Diana (Finance dept) => calculate average of Finance => compare Diana's salary

The inner query needs to change its calculation based on which employee row we are currently checking. That is a Correlated Subquery.

```
SELECT e1.NAME, e1.DEPARTMENT, e1.SALARY
FROM EMPLOYEES e1
WHERE e1.SALARY > (SELECT AVG(e2.SALARY)
                   FROM EMPLOYEES e2
                   WHERE e2.DEPARTMENT = e1.DEPARTMENT)
```

```
WHERE e2.DEPARTMENT = e1.DEPARTMENT) ;
|__ References outer query
alias 'e1' __|
```

Step-by-Step Explanation

26. Outer query picks up Alice (HR, 50000)
27. Inner query: SELECT AVG(SALARY) FROM EMPLOYEES WHERE DEPARTMENT = 'HR' => (50000+48000)/2 = 49000
28. 50000 > 49000 => TRUE => Alice is included
29. Outer query picks up Bob (IT, 80000)
30. Inner query: AVG of IT dept = (80000+75000+82000)/3 = 79000
31. 80000 > 79000 => TRUE => Bob is included
32. Charlie (IT, 75000): 75000 > 79000 => FALSE => Charlie is excluded
33. This repeats for every employee row.

Expected Output:

NAME	DEPARTMENT	SALARY
Alice	HR	50000
Bob	IT	80000
Diana	Finance	90000
Frank	Finance	95000
Grace	IT	82000

Key Identifier: You can identify a correlated subquery because the inner query uses a column from the outer query's alias (e1.DEPARTMENT is from the outer query).

Step 10: Practice Examples on EMPLOYEES Table

EMP_ID	NAME	DEPARTMENT	SALARY	MANAGER_ID
101	Alice	HR	50000	NULL
102	Bob	IT	80000	101
103	Charlie	IT	75000	101
104	Diana	Finance	90000	NULL
105	Eve	HR	48000	101
106	Frank	Finance	95000	104
107	Grace	IT	82000	101

Example 1: Find the second highest salary

Question: Who earns the second highest salary in the company?

```
SELECT MAX(SALARY) AS SECOND_HIGHEST_SALARY
FROM EMPLOYEES
WHERE SALARY < (SELECT MAX(SALARY) FROM EMPLOYEES);
```

Explanation: Inner query finds the maximum salary (95000). Outer query finds the maximum among salaries less than 95000.

Expected Output:

SECOND_HIGHEST_SALARY
90000

Example 2: Find employees who have no subordinates (not a manager)

Question: Which employees are not managing anyone?

```
SELECT NAME
FROM EMPLOYEES
WHERE EMP_ID NOT IN (SELECT DISTINCT MANAGER_ID FROM EMPLOYEES
                     WHERE MANAGER_ID IS NOT NULL);
```

Inner query returns: 101, 104 (the manager IDs). NOT IN returns employees NOT in this list.

Expected Output:

NAME
Bob
Charlie
Eve
Frank
Grace

Example 3: Find departments where all employees earn above 70000

```
SELECT DISTINCT DEPARTMENT
FROM EMPLOYEES e1
WHERE 70000 < ALL (SELECT SALARY FROM EMPLOYEES e2
                  WHERE e2.DEPARTMENT = e1.DEPARTMENT) ;
```

For each department, the inner query returns all salaries of that department. 70000 < ALL means all salaries must be above 70000.

Expected Output:

DEPARTMENT
Finance

Example 4: Find employees earning between the min and max salary of IT dept

```
SELECT NAME, SALARY
FROM EMPLOYEES
WHERE SALARY BETWEEN (SELECT MIN(SALARY) FROM EMPLOYEES WHERE DEPARTMENT
= 'IT')
AND (SELECT MAX(SALARY) FROM EMPLOYEES WHERE DEPARTMENT
= 'IT') ;
```

First subquery: MIN salary in IT = 75000. Second subquery: MAX salary in IT = 82000.

Expected Output:

NAME	SALARY
Bob	80000
Charlie	75000
Grace	82000

Step 11: Practice Questions with Hints and Solutions

Table Used: EMPLOYEES (same table as before)

EMP_ID	NAME	DEPARTMENT	SALARY	MANAGER_ID
101	Alice	HR	50000	NULL
102	Bob	IT	80000	101
103	Charlie	IT	75000	101
104	Diana	Finance	90000	NULL
105	Eve	HR	48000	101
106	Frank	Finance	95000	104
107	Grace	IT	82000	101

Question 1: Find all employees who earn more than the average salary of the company.

Hint: Think: First, what is the average salary of ALL employees? You need one calculated value. Then compare each employee's salary to that one value. Which operator: = or > ?

Answer / Solution:

```
SELECT NAME, SALARY
FROM EMPLOYEES
WHERE SALARY > (SELECT AVG(SALARY) FROM EMPLOYEES);

-- Inner query calculates AVG(SALARY) = 74285.71
-- Outer query filters employees where salary > 74285.71
```

Expected Output:

NAME	SALARY
Bob	80000
Diana	90000
Frank	95000
Grace	82000

Question 2: Find employees from the same department as 'Bob'.

Hint: Think: You don't know Bob's department number in your head. SQL must first find Bob's department, then return all employees in that same department. Is it one value or many values from the inner query?

Answer / Solution:

```
SELECT NAME, DEPARTMENT
```



```
FROM EMPLOYEES
WHERE DEPARTMENT = (SELECT DEPARTMENT FROM EMPLOYEES WHERE NAME = 'Bob')
AND NAME <> 'Bob';

-- Inner query: returns 'IT'
-- Outer query: finds all IT employees except Bob himself
```

Expected Output:

NAME	DEPARTMENT
Charlie	IT
Grace	IT

Question 3: Find employees whose salary is equal to any employee's salary in the HR department.

Hint: Think: HR department has multiple employees, so multiple salaries. The inner query returns a LIST. You cannot use = with a list. Which operator handles a list: IN, ANY, or ALL?

Answer / Solution:

```
SELECT NAME, SALARY
FROM EMPLOYEES
WHERE SALARY IN (SELECT SALARY FROM EMPLOYEES WHERE DEPARTMENT = 'HR');

-- Inner query returns: 50000, 48000
-- Outer query finds employees with salary IN (50000, 48000)
```

Expected Output:

NAME	SALARY
Alice	50000
Eve	48000

Question 4: Find employees who earn more than their own department's average salary.

Hint: Think: 'their own department average' changes per employee. For each employee row, the average must be recalculated for THAT department. Does a regular subquery work here? Or does the inner query need to 'know' which row the outer query is looking at?

Answer / Solution:

```
SELECT e1.NAME, e1.DEPARTMENT, e1.SALARY
FROM EMPLOYEES e1
WHERE e1.SALARY > (SELECT AVG(e2.SALARY)
                   FROM EMPLOYEES e2
                   WHERE e2.DEPARTMENT = e1.DEPARTMENT);

-- This is a Correlated Subquery
```

```
-- e1.DEPARTMENT connects the inner and outer query
```

Expected Output:

NAME	DEPARTMENT	SALARY
Alice	HR	50000
Bob	IT	80000
Diana	Finance	90000
Frank	Finance	95000
Grace	IT	82000

Question 5: Find the employee(s) with the minimum salary in each department.

Hint: Think: For each employee, you need to check if their salary is equal to the minimum salary of THEIR OWN department. The minimum changes per department. Sounds like you need the inner query to depend on the current row's department.

Answer / Solution:

```
SELECT e1.NAME, e1.DEPARTMENT, e1.SALARY
FROM EMPLOYEES e1
WHERE e1.SALARY = (SELECT MIN(e2.SALARY)
                  FROM EMPLOYEES e2
                  WHERE e2.DEPARTMENT = e1.DEPARTMENT) ;
```

Expected Output:

NAME	DEPARTMENT	SALARY
Eve	HR	48000
Charlie	IT	75000
Diana	Finance	90000

Step 12: EXISTS and IN Operator — Detailed Explanation

The IN Operator

IN checks whether a value exists in a LIST of values returned by a subquery.

```
SELECT column FROM table
WHERE column IN (SELECT column FROM another_table WHERE condition);
```

How IN works:

34. The inner query runs first and returns a list of values.
35. The outer query checks if each row's column value is present in that list.
36. If yes, the row is included in the result.

Key Point about IN: IN collects ALL results from the subquery into memory first, then checks against the outer query. If the subquery returns a large number of rows, this can be slow.

The EXISTS Operator

EXISTS checks whether a subquery RETURNS ANY ROWS AT ALL. It does not care about the actual values. It only asks: 'Did the subquery find at least one matching row?'

```
SELECT column FROM table t1
WHERE EXISTS (SELECT 1 FROM another_table t2 WHERE t2.column = t1.column);
```

How EXISTS works:

37. Outer query picks up a row from t1.
38. Inner query runs with the reference to that t1 row.
39. If the inner query returns at least one row, EXISTS returns TRUE and the outer row is included.
40. If the inner query returns no rows, EXISTS returns FALSE.
41. EXISTS stops searching as soon as it finds ONE matching row (short-circuit). This makes it fast.

IN vs EXISTS: Side-by-Side Comparison

Point	IN	EXISTS
What it checks	Whether a value is in the list	Whether any matching row exists
Subquery type	Non-correlated (usually)	Correlated (usually)
Stops early?	No, fetches all values first	Yes, stops at first match
Best for	Small to medium result sets	Large tables, checking existence only

Point	IN	EXISTS
Handles NULLs	Tricky — IN with NULLs can fail	EXISTS handles NULLs safely
SELECT in inner	SELECT actual column	SELECT 1 or SELECT * (value not used)

Quick Decision Rule: Use IN when you want to compare against a known list. Use EXISTS when you only need to know IF something exists (especially with large tables or NULLs).

Step 13: EXISTS and IN — Examples

Tables Used: EMPLOYEES and DEPARTMENTS

EMPLOYEES:

EMP_ID	NAME	DEPARTMENT	SALARY	MANAGER_ID
101	Alice	HR	50000	NULL
102	Bob	IT	80000	101
103	Charlie	IT	75000	101
104	Diana	Finance	90000	NULL
105	Eve	HR	48000	101
106	Frank	Finance	95000	104
107	Grace	IT	82000	101

DEPARTMENTS:

DEPT_ID	DEPT_NAME	LOCATION
10	HR	Mumbai
20	IT	Bangalore
30	Finance	Delhi

EXISTS Example 1

Question: Find all departments that have at least one employee earning more than 80000.

Hint: You only need to check IF such an employee EXISTS in the department. You do not need the actual salary. This is a perfect case for EXISTS.

```
SELECT DEPT_NAME
FROM DEPARTMENTS d
WHERE EXISTS (SELECT 1
              FROM EMPLOYEES e
```

```
WHERE e.DEPARTMENT = d.DEPT_NAME
AND e.SALARY > 80000);
```

How it works: For each department row, SQL checks if there is any employee in that department earning > 80000. If yes, the department row is returned.

Expected Output:

DEPT_NAME
IT
Finance

EXISTS Example 2

Question: Find all employees who have at least one other colleague in the same department.

Hint: For each employee, check if there EXISTS another employee (different EMP_ID) in the same department. Correlated EXISTS is ideal here.

```
SELECT e1.NAME, e1.DEPARTMENT
FROM EMPLOYEES e1
WHERE EXISTS (SELECT 1
              FROM EMPLOYEES e2
              WHERE e2.DEPARTMENT = e1.DEPARTMENT
              AND e2.EMP_ID <> e1.EMP_ID);
```

Expected Output:

NAME	DEPARTMENT
Alice	HR
Bob	IT
Charlie	IT
Diana	Finance
Eve	HR
Frank	Finance
Grace	IT

EXISTS Example 3 (NOT EXISTS)

Question: Find departments that have NO employees.

Hint: If a department has no employees, the EXISTS check will return FALSE. Use NOT EXISTS to find those departments.

```
SELECT DEPT_NAME
FROM DEPARTMENTS d
WHERE NOT EXISTS (SELECT 1
```

```
FROM EMPLOYEES e
WHERE e.DEPARTMENT = d.DEPT_NAME);
```

In our current data, all departments have employees, so no rows are returned. If you add a new department 'Marketing' with no employees, it would appear here.

Expected Output:

DEPT_NAME
(No rows returned - all depts have employees)

IN Example 1

Question: Find employees who work in departments located in Bangalore.

Hint: You need to first get a list of department names located in Bangalore, then find employees IN those departments. IN is appropriate here because you are comparing against a list.

```
SELECT NAME, DEPARTMENT
FROM EMPLOYEES
WHERE DEPARTMENT IN (SELECT DEPT_NAME FROM DEPARTMENTS
                     WHERE LOCATION = 'Bangalore');
```

Inner query returns: IT (only Bangalore dept)

Expected Output:

NAME	DEPARTMENT
Bob	IT
Charlie	IT
Grace	IT

IN Example 2

Question: Find employees who are NOT from departments located in Mumbai.

Hint: Use NOT IN to exclude employees in departments from Mumbai. NOT IN reverses the filter.

```
SELECT NAME, DEPARTMENT
FROM EMPLOYEES
WHERE DEPARTMENT NOT IN (SELECT DEPT_NAME FROM DEPARTMENTS
                        WHERE LOCATION = 'Mumbai');
```

Mumbai departments: HR. NOT IN means exclude HR employees.

Expected Output:

NAME	DEPARTMENT
Bob	IT

NAME	DEPARTMENT
Charlie	IT
Diana	Finance
Frank	Finance
Grace	IT

IN Example 3

Question: Find employees whose EMP_ID appears as a MANAGER_ID for other employees.

Hint: You need a list of manager IDs from the same table, then find employees whose EMP_ID is IN that list.

```
SELECT NAME, EMP_ID
FROM EMPLOYEES
WHERE EMP_ID IN (SELECT DISTINCT MANAGER_ID FROM EMPLOYEES
WHERE MANAGER_ID IS NOT NULL) ;
```

Expected Output:

NAME	EMP_ID
Alice	101
Diana	104

Step 14: Which is Faster — EXISTS or IN?

This is one of the most common interview questions in SQL. The answer is: it depends on the situation. But once you understand how each operator works internally, you will always know which to pick.

First, Understand What Each Operator Does Internally

How IN Works Internally

1. SQL runs the ENTIRE inner query first.
2. All returned values are collected into a list in memory.
3. Only then does the outer query run, checking each row against that full list.
4. IN does NOT stop early. It always loads every value from the subquery.

How EXISTS Works Internally

1. Outer query picks up the first row.

2. Inner query runs for that row only.
3. The moment it finds ONE matching row, EXISTS returns TRUE and stops searching. It does not scan the rest.
4. This early exit is called SHORT-CIRCUIT EVALUATION. It saves a lot of time when the inner table is large.

A Simple Real-Life Analogy

Story: You are looking for a specific book in a library. **IN approach:** You ask the librarian to give you a complete printed list of ALL book titles first. Then you search through the entire list yourself. Even if the book you want is the very first one, you waited for the full list. **EXISTS approach:** You walk directly to the shelves. The moment you spot the book, you stop looking. You never checked the remaining shelves. Much faster when the library is huge.

The Three Scenarios — When Each is Faster

Situation	Use IN	Use EXISTS
Subquery returns a SMALL number of rows (e.g., 10-100 rows)	Fast. The list is small. Loading it into memory and checking is quick.	Acceptable, but the short-circuit advantage is small when the list is already tiny.
Subquery returns a LARGE number of rows (e.g., 100,000+ rows)	Slower. Entire large result loaded into memory first. Can cause performance issues.	Much faster. Stops the moment it finds the first match. Does not load everything.
Outer table is SMALL , inner table is LARGE	Slower — inner query must fully complete.	Faster — EXISTS short-circuits on first match per outer row.
Outer table is LARGE , inner table is SMALL	Faster — small list is loaded once and all outer rows are checked quickly.	Acceptable — but no significant speed advantage over IN here.
Inner query result contains NULL values	Dangerous — IN with NULLs can return unexpected results or no rows at all.	Safe — EXISTS only checks for row existence, NULLs do not affect it.

The NULL Problem with IN — Must Know

This is a critical difference that most beginners miss. When the inner subquery returns even ONE NULL value, IN can silently return zero rows.

Example: Suppose `MANAGER_ID` has a NULL value in some rows.

```
-- This query may return NO rows if MANAGER_ID has any NULL
SELECT NAME FROM EMPLOYEES
WHERE EMP_ID NOT IN (SELECT MANAGER_ID FROM EMPLOYEES);

-- Safe version — always filter out NULLs when using IN
```



```
SELECT NAME FROM EMPLOYEES
WHERE EMP_ID NOT IN (SELECT MANAGER_ID FROM EMPLOYEES
                     WHERE MANAGER_ID IS NOT NULL);
```

Why this happens: SQL uses three-valued logic: TRUE, FALSE, and UNKNOWN. When you compare any value to NULL using IN, the result is UNKNOWN (not FALSE). NOT IN with any NULL in the list makes the entire WHERE condition UNKNOWN, which means no rows pass. EXISTS does not have this problem because it never compares values directly — it only checks if a row exists.

Side-by-Side Query Comparison

Same question answered with both operators so you can see the difference in structure.

Question: Find employees who work in a department that exists in the DEPARTMENTS table.

Using IN:

```
SELECT NAME FROM EMPLOYEES
WHERE DEPARTMENT IN (SELECT DEPT_NAME FROM DEPARTMENTS);

-- Inner query runs ONCE, returns all DEPT_NAMES: HR, IT, Finance
-- Outer query checks each employee against that full list
```

Using EXISTS:

```
SELECT e.NAME FROM EMPLOYEES e
WHERE EXISTS (SELECT 1 FROM DEPARTMENTS d
              WHERE d.DEPT_NAME = e.DEPARTMENT);

-- For each employee row, inner query checks if their dept exists in
DEPARTMENTS
-- Stops the moment it finds a match (short-circuit)
```

Note on SELECT 1: Inside EXISTS, you write SELECT 1 (or SELECT *). The actual value does not matter. EXISTS only cares whether any row was found, not what that row contains. Writing SELECT 1 makes it clear to anyone reading the query that the value is intentionally ignored.

Final Decision Guide

Choose IN when...	Choose EXISTS when...
Subquery returns a small, known set of values	Subquery could return a large number of rows
You are comparing values from a lookup/reference table	You only need to check whether a related row exists

Choose IN when...	Choose EXISTS when...
You are confident there are no NULLs in the subquery result	The subquery may return NULLs (EXISTS handles them safely)
The query is simple and readability matters more	Performance is critical and tables are large
You use IN with a static list: WHERE id IN (1, 2, 3)	You need a correlated check per row of the outer query

One Line Summary: IN is convenient for small lists. EXISTS is safer and faster for large tables, correlated checks, and when NULLs might be present. When in doubt on large datasets, prefer EXISTS.



Step 15: 20 Practice Questions on Subqueries

New Sample Tables

We will use three tables for these exercises.

Table 1: ORDERS

ORDER_ID	CUSTOMER_ID	PRODUCT_ID	QUANTITY	ORDER_DATE	STATUS
1001	C01	P01	2	2024-01-05	Delivered
1002	C02	P02	1	2024-01-10	Pending
1003	C01	P03	5	2024-01-12	Delivered
1004	C03	P01	3	2024-01-15	Cancelled
1005	C02	P04	2	2024-02-01	Delivered
1006	C04	P02	1	2024-02-05	Pending
1007	C03	P03	4	2024-02-10	Delivered

Table 2: CUSTOMERS

CUSTOMER_ID	CUSTOMER_NAME	CITY	CREDIT_LIMIT
C01	Ravi Kumar	Delhi	100000
C02	Priya Singh	Mumbai	80000
C03	Amit Sharma	Bangalore	60000
C04	Neha Joshi	Chennai	50000

Table 3: PRODUCTS

PRODUCT_ID	PRODUCT_NAME	CATEGORY	PRICE
P01	Laptop	Electronics	55000
P02	Headphones	Electronics	3000
P03	Desk Chair	Furniture	12000
P04	Notebook	Stationery	150

Note: Read each question carefully. Think about: (1) What do I need to find first? (2) Is the inner result one value or many values? (3) Do I need EXISTS or IN? Then write your query and match with the solution at the end.

Question 1: Find the names of customers who have placed at least one order.

Hint: Think: Get a list of CUSTOMER_IDs that appear in the ORDERS table, then find customers IN that list.

Question 2: Find products that have NEVER been ordered.

Hint: Think: Get all ordered PRODUCT_IDs from ORDERS, then find products NOT IN that list.

Question 3: Find customers whose credit limit is above the average credit limit of all customers.

Hint: Think: Average credit limit is one single value. Which operator compares one value?

Question 4: Find the most expensive product.

Hint: Think: MAX(PRICE) gives one value. Compare each product price to that one value using =.

Question 5: Find orders where the product's price is above 10000.

Hint: Think: First get product IDs with price > 10000 from PRODUCTS, then find orders where PRODUCT_ID is IN that list.

Question 6: Find customers who have placed more than 2 orders.

Hint: Think: Group orders by CUSTOMER_ID and count. Use HAVING COUNT > 2. Return those CUSTOMER_IDs. Use IN to match with CUSTOMERS table.

Question 7: Find all 'Delivered' orders where the product belongs to the 'Electronics' category.

Hint: Think: Two filters needed. First get Electronics PRODUCT_IDs, then filter ORDERS for STATUS = Delivered AND PRODUCT_ID IN that list.

Question 8: Find the customer who placed the maximum number of orders.

Hint: Think: Group ORDERS by CUSTOMER_ID, count orders, find the max count. Use a nested subquery.

Question 9: Find products whose price is greater than ALL products in the 'Stationery' category.

Hint: Think: Get prices of all Stationery products. Use > ALL to ensure the product price beats every Stationery price.

Question 10: Find customers who have NOT placed any orders with status 'Cancelled'.

Hint: Think: Get CUSTOMER_IDs from ORDERS where STATUS = Cancelled. Use NOT IN to exclude them from CUSTOMERS.

Question 11: Find orders placed by customers from 'Delhi'.

Hint: Think: Get CUSTOMER_IDs of Delhi customers first, then find ORDERS where CUSTOMER_ID is IN that list.

Question 12: Find products that have been ordered with a quantity greater than 3 at least once.

Hint: Think: Get PRODUCT_IDs from ORDERS where QUANTITY > 3. Use IN to find matching products.

Question 13: Find all customers whose credit limit is less than the credit limit of 'Ravi Kumar'.

Hint: Think: First find Ravi Kumar's credit limit (one value). Then compare other customers to that value using <.

Question 14: Find customers who have placed orders for products in the 'Furniture' category.

Hint: Think: Get Furniture PRODUCT_IDs first. Then get CUSTOMER_IDs from ORDERS where PRODUCT_ID is in that list. Then find customer names using IN.

Question 15: Find the second most expensive product.

Hint: Think: Find the MAX price first. Then find the MAX price that is less than that maximum.

Question 16: Find customers who exist in the CUSTOMERS table but have no record in the ORDERS table. (Use EXISTS)

Hint: Think: For each customer, check if there EXISTS any order for that customer. Use NOT EXISTS to find those without orders.

Question 17: Find products that were ordered more total quantity than the average total quantity ordered per product.

Hint: Think: First calculate average total quantity per product using GROUP BY. Then compare each product's total quantity to that average.

Question 18: Find orders where the ordered quantity is greater than the average quantity of orders for that same product. (Correlated)

Hint: Think: For each order row, calculate the average quantity for that specific PRODUCT_ID. Compare the order's quantity to that product's average. This changes per product.

Question 19: Find customers who have placed orders for every product that costs more than 10000.

Hint: Think: This is an 'all matching' problem. Get products with price > 10000 first. Then find customers whose orders cover all those products.

Question 20: Find products that have been ordered by at least 2 different customers.

Hint: Think: Group ORDERS by PRODUCT_ID and count DISTINCT CUSTOMER_IDs. Use HAVING COUNT >= 2. Match with PRODUCTS using IN.

Step 16: Solutions to All 20 Questions

Solution 1: Customers who have placed at least one order

```
SELECT CUSTOMER_NAME FROM CUSTOMERS
WHERE CUSTOMER_ID IN (SELECT DISTINCT CUSTOMER_ID FROM ORDERS) ;
```

Expected Output:

CUSTOMER_NAME
Ravi Kumar
Priya Singh
Amit Sharma
Neha Joshi

Solution 2: Products never ordered

```
SELECT PRODUCT_NAME FROM PRODUCTS
WHERE PRODUCT_ID NOT IN (SELECT DISTINCT PRODUCT_ID FROM ORDERS) ;
```

Expected Output:

PRODUCT_NAME
(All products have been ordered in our data)

Solution 3: Customers with above average credit limit

```
SELECT CUSTOMER_NAME, CREDIT_LIMIT FROM CUSTOMERS
WHERE CREDIT_LIMIT > (SELECT AVG(CREDIT_LIMIT) FROM CUSTOMERS) ;
```

AVG CREDIT_LIMIT = (100000+80000+60000+50000)/4 = 72500

Expected Output:

CUSTOMER_NAME	CREDIT_LIMIT
Ravi Kumar	100000
Priya Singh	80000

Solution 4: Most expensive product

```
SELECT PRODUCT_NAME, PRICE FROM PRODUCTS
WHERE PRICE = (SELECT MAX(PRICE) FROM PRODUCTS) ;
```

Expected Output:

PRODUCT_NAME	PRICE
Laptop	55000

Solution 5: Orders with product price above 10000

```
SELECT ORDER_ID, PRODUCT_ID, STATUS FROM ORDERS
WHERE PRODUCT_ID IN (SELECT PRODUCT_ID FROM PRODUCTS WHERE PRICE >
10000) ;
```

Products with price > 10000: Laptop (P01, 55000), Desk Chair (P03, 12000)

Expected Output:

ORDER_ID	PRODUCT_ID	STATUS
1001	P01	Delivered
1003	P03	Delivered
1004	P01	Cancelled
1007	P03	Delivered

Solution 6: Customers who placed more than 2 orders

```
SELECT CUSTOMER_NAME FROM CUSTOMERS
WHERE CUSTOMER_ID IN (SELECT CUSTOMER_ID FROM ORDERS
GROUP BY CUSTOMER_ID HAVING COUNT(*) > 2) ;
```

Order counts: C01 => 2, C02 => 2, C03 => 2, C04 => 1. No customer placed more than 2 orders in our sample data.

Expected Output:

CUSTOMER_NAME
(No rows — no customer has more than 2 orders in sample)

Solution 7: Delivered orders for Electronics products

```
SELECT ORDER_ID, CUSTOMER_ID, PRODUCT_ID FROM ORDERS
WHERE STATUS = 'Delivered'
AND PRODUCT_ID IN (SELECT PRODUCT_ID FROM PRODUCTS WHERE CATEGORY =
'Electronics');
```

Electronics: P01 (Laptop), P02 (Headphones)

Expected Output:

ORDER_ID	CUSTOMER_ID	PRODUCT_ID
1001	C01	P01
1005	C02	P04

Solution 8: Customer with maximum number of orders

```
SELECT CUSTOMER_NAME FROM CUSTOMERS
WHERE CUSTOMER_ID = (SELECT CUSTOMER_ID FROM ORDERS
```

```
GROUP BY CUSTOMER_ID
ORDER BY COUNT(*) DESC LIMIT 1);
```

All customers have 2 orders each. Result could be any one of them (first returned by the DB).

Expected Output:

CUSTOMER_NAME
Ravi Kumar (or Priya Singh or Amit Sharma)

Solution 9: Products priced higher than ALL Stationery products

```
SELECT PRODUCT_NAME, PRICE FROM PRODUCTS
WHERE PRICE > ALL (SELECT PRICE FROM PRODUCTS WHERE CATEGORY =
'Stationery');
```

Stationery max price: 150. Products with price > 150: Laptop, Headphones, Desk Chair.

Expected Output:

PRODUCT_NAME	PRICE
Laptop	55000
Headphones	3000
Desk Chair	12000

Solution 10: Customers with no cancelled orders

```
SELECT CUSTOMER_NAME FROM CUSTOMERS
WHERE CUSTOMER_ID NOT IN (SELECT DISTINCT CUSTOMER_ID FROM ORDERS WHERE
STATUS = 'Cancelled');
```

C03 placed a cancelled order (ORDER_ID 1004). So C03 (Amit Sharma) is excluded.

Expected Output:

CUSTOMER_NAME
Ravi Kumar
Priya Singh
Neha Joshi

Solution 11: Orders placed by Delhi customers

```
SELECT ORDER_ID, CUSTOMER_ID, STATUS FROM ORDERS
WHERE CUSTOMER_ID IN (SELECT CUSTOMER_ID FROM CUSTOMERS WHERE CITY =
'Delhi');
```

Delhi customer: Ravi Kumar (C01)

Expected Output:

ORDER_ID	CUSTOMER_ID	STATUS
1001	C01	Delivered
1003	C01	Delivered

Solution 12: Products ordered with quantity > 3 at least once

```
SELECT PRODUCT_NAME FROM PRODUCTS
WHERE PRODUCT_ID IN (SELECT DISTINCT PRODUCT_ID FROM ORDERS WHERE
QUANTITY > 3);
```

Orders with qty > 3: ORDER 1003 (P03, qty 5), ORDER 1007 (P03, qty 4)

Expected Output:

PRODUCT_NAME
Desk Chair

Solution 13: Customers with credit limit less than Ravi Kumar's

```
SELECT CUSTOMER_NAME, CREDIT_LIMIT FROM CUSTOMERS
WHERE CREDIT_LIMIT < (SELECT CREDIT_LIMIT FROM CUSTOMERS WHERE
CUSTOMER_NAME = 'Ravi Kumar');
```

Ravi Kumar's credit limit: 100000

Expected Output:

CUSTOMER_NAME	CREDIT_LIMIT
Priya Singh	80000
Amit Sharma	60000
Neha Joshi	50000

Solution 14: Customers who ordered Furniture products

```
SELECT CUSTOMER_NAME FROM CUSTOMERS
WHERE CUSTOMER_ID IN (SELECT DISTINCT CUSTOMER_ID FROM ORDERS
WHERE PRODUCT_ID IN (SELECT PRODUCT_ID FROM
PRODUCTS WHERE CATEGORY = 'Furniture'));
```

Furniture: P03 (Desk Chair). Orders for P03: C01 (ord 1003), C03 (ord 1007).

Expected Output:

CUSTOMER_NAME
Ravi Kumar
Amit Sharma

Solution 15: Second most expensive product

```
SELECT PRODUCT_NAME, PRICE FROM PRODUCTS
WHERE PRICE = (SELECT MAX(PRICE) FROM PRODUCTS
              WHERE PRICE < (SELECT MAX(PRICE) FROM PRODUCTS));
```

Max price: 55000 (Laptop). Second max: 12000 (Desk Chair).

Expected Output:

PRODUCT_NAME	PRICE
Desk Chair	12000

Solution 16: Customers with no orders (NOT EXISTS)

```
SELECT CUSTOMER_NAME FROM CUSTOMERS c
WHERE NOT EXISTS (SELECT 1 FROM ORDERS o WHERE o.CUSTOMER_ID =
c.CUSTOMER_ID);
```

All customers in our sample data have at least one order.

Expected Output:

CUSTOMER_NAME
(No rows — all customers have orders)

Solution 17: Products ordered more than average total quantity

```
SELECT PRODUCT_NAME FROM PRODUCTS
WHERE PRODUCT_ID IN (
  SELECT PRODUCT_ID FROM ORDERS
  GROUP BY PRODUCT_ID
  HAVING SUM(QUANTITY) > (SELECT AVG(total_qty) FROM (SELECT PRODUCT_ID,
SUM(QUANTITY) AS total_qty FROM ORDERS GROUP BY PRODUCT_ID) t)
);
```

Total per product: P01=5, P02=2, P03=9, P04=2. Average = 18/4 = 4.5. Products above 4.5: P01 (5), P03 (9).

Expected Output:

PRODUCT_NAME
Laptop
Desk Chair

Solution 18: Orders with quantity above average for that product (Correlated)

```
SELECT o1.ORDER_ID, o1.PRODUCT_ID, o1.QUANTITY
FROM ORDERS o1
WHERE o1.QUANTITY > (SELECT AVG(o2.QUANTITY) FROM ORDERS o2
                     WHERE o2.PRODUCT_ID = o1.PRODUCT_ID);
```

For each order, compare its quantity to the avg quantity for that product. Correlated subquery runs once per order row.

Expected Output:

ORDER_ID	PRODUCT_ID	QUANTITY
1001	P01	2
1003	P03	5
1007	P03	4

Solution 19: Customers who ordered all products priced above 10000

```
-- Products priced > 10000: P01 (Laptop, 55000), P03 (Desk Chair, 12000)
SELECT CUSTOMER_NAME FROM CUSTOMERS c
WHERE NOT EXISTS (
    SELECT 1 FROM PRODUCTS p WHERE p.PRICE > 10000
    AND NOT EXISTS (SELECT 1 FROM ORDERS o WHERE o.CUSTOMER_ID =
c.CUSTOMER_ID AND o.PRODUCT_ID = p.PRODUCT_ID)
);
```

C01 (Ravi Kumar) ordered both P01 (Order 1001) and P03 (Order 1003). So Ravi Kumar qualifies.

Expected Output:

CUSTOMER_NAME
Ravi Kumar

Solution 20: Products ordered by at least 2 different customers

```
SELECT PRODUCT_NAME FROM PRODUCTS
WHERE PRODUCT_ID IN (SELECT PRODUCT_ID FROM ORDERS
    GROUP BY PRODUCT_ID
    HAVING COUNT(DISTINCT CUSTOMER_ID) >= 2);
```

P01: ordered by C01, C03 (2 customers). P03: ordered by C01, C03 (2 customers). P02: ordered by C02, C04 (2 customers). P04: ordered by C02 only (1 customer).

Expected Output:

PRODUCT_NAME
Laptop
Headphones
Desk Chair

Appendix: MySQL Table Structure and Sample Data

This appendix provides the complete MySQL SQL scripts to create all tables used in this guide and insert the exact sample data. Run these scripts in order before practising any query from this document.

How to Use: Open MySQL Workbench or any MySQL client. Create a new database (schema), select it, then run each script block below in the order shown.

Step 0: Create and Select the Database

```
-- Create a dedicated practice database
CREATE DATABASE IF NOT EXISTS sql_practice;
USE sql_practice;
```

Table Set 1 — Used in Steps 1 to 13

These two tables (EMPLOYEES and DEPARTMENTS) are used for all examples in Steps 1 through 13, including subquery types, IN / ANY / ALL, correlated subqueries, and EXISTS vs IN.

Table 1 of 2 — DEPARTMENTS

Preview of data:

DEPT_ID	DEPT_NAME	LOCATION
10	HR	Mumbai
20	IT	Bangalore
30	Finance	Delhi

CREATE TABLE — DEPARTMENTS

```
-- Drop the table first if it already exists (safe re-run)
DROP TABLE IF EXISTS DEPARTMENTS;

CREATE TABLE DEPARTMENTS (
  DEPT_ID      INT          NOT NULL AUTO_INCREMENT,
  DEPT_NAME    VARCHAR(50)  NOT NULL,
  LOCATION     VARCHAR(100) NOT NULL,
  PRIMARY KEY (DEPT_ID)
);
```

INSERT Data — DEPARTMENTS

```
-- Insert all department rows
INSERT INTO DEPARTMENTS (DEPT_NAME, LOCATION) VALUES
  ('HR',      'Mumbai'),
  ('IT',      'Bangalore'),
  ('Finance', 'Delhi');
```

Table 2 of 2 — EMPLOYEES

Preview of data:

EMP_ID	NAME	DEPARTMENT	SALARY	MANAGER_ID
101	Alice	HR	50000	NULL
102	Bob	IT	80000	101
103	Charlie	IT	75000	101
104	Diana	Finance	90000	NULL
105	Eve	HR	48000	101
106	Frank	Finance	95000	104
107	Grace	IT	82000	101

CREATE TABLE — EMPLOYEES

```
-- DEPARTMENTS must be created before EMPLOYEES (foreign key dependency)
DROP TABLE IF EXISTS EMPLOYEES;

CREATE TABLE EMPLOYEES (
    EMP_ID      INT          NOT NULL,
    NAME        VARCHAR(100) NOT NULL,
    DEPARTMENT  VARCHAR(50)  NOT NULL,
    SALARY      DECIMAL(10,2) NOT NULL,
    MANAGER_ID  INT          NULL,
    PRIMARY KEY (EMP_ID),
    FOREIGN KEY (MANAGER_ID) REFERENCES EMPLOYEES (EMP_ID)
);
```

INSERT Data — EMPLOYEES

```
-- Top-level managers (no MANAGER_ID) must be inserted first
INSERT INTO EMPLOYEES (EMP_ID, NAME, DEPARTMENT, SALARY, MANAGER_ID) VALUES
    (101, 'Alice', 'HR', 50000.00, NULL),
    (104, 'Diana', 'Finance', 90000.00, NULL);

-- Employees who report to Alice (101) or Diana (104)
INSERT INTO EMPLOYEES (EMP_ID, NAME, DEPARTMENT, SALARY, MANAGER_ID) VALUES
    (102, 'Bob', 'IT', 80000.00, 101),
    (103, 'Charlie', 'IT', 75000.00, 101),
    (105, 'Eve', 'HR', 48000.00, 101),
    (106, 'Frank', 'Finance', 95000.00, 104),
    (107, 'Grace', 'IT', 82000.00, 101);
```

Verify the Data

```
-- Run these SELECT statements to confirm data is inserted correctly
```

```
SELECT * FROM DEPARTMENTS;
SELECT * FROM EMPLOYEES;
SELECT * FROM EMPLOYEES ORDER BY EMP_ID;
```

Note on MANAGER_ID: Alice (101) and Diana (104) have NULL in MANAGER_ID because they are top-level managers. The self-referencing foreign key means MANAGER_ID points back to the same EMPLOYEES table. That is why Alice and Diana must be inserted first — otherwise MySQL will reject the foreign key reference.

Table Set 2 — Used in Step 14 (20 Practice Questions)

These three tables (CUSTOMERS, PRODUCTS, and ORDERS) are used exclusively for the 20 practice questions in Step 14. Create them in the order shown below: CUSTOMERS and PRODUCTS first (no dependencies), then ORDERS last (depends on both).

Table 1 of 3 — CUSTOMERS

Preview of data:

CUSTOMER_ID	CUSTOMER_NAME	CITY	CREDIT_LIMIT
C01	Ravi Kumar	Delhi	100000
C02	Priya Singh	Mumbai	80000
C03	Amit Sharma	Bangalore	60000
C04	Neha Joshi	Chennai	50000

CREATE TABLE — CUSTOMERS

```
DROP TABLE IF EXISTS CUSTOMERS;

CREATE TABLE CUSTOMERS (
    CUSTOMER_ID    VARCHAR(10)    NOT NULL,
    CUSTOMER_NAME  VARCHAR(100)   NOT NULL,
    CITY           VARCHAR(100)   NOT NULL,
    CREDIT_LIMIT   DECIMAL(10,2)  NOT NULL,
    PRIMARY KEY (CUSTOMER_ID)
);
```

INSERT Data — CUSTOMERS

```
INSERT INTO CUSTOMERS (CUSTOMER_ID, CUSTOMER_NAME, CITY, CREDIT_LIMIT) VALUES
('C01', 'Ravi Kumar', 'Delhi', 100000.00),
('C02', 'Priya Singh', 'Mumbai', 80000.00),
('C03', 'Amit Sharma', 'Bangalore', 60000.00),
('C04', 'Neha Joshi', 'Chennai', 50000.00);
```


Table 2 of 3 — PRODUCTS

Preview of data:

PRODUCT_ID	PRODUCT_NAME	CATEGORY	PRICE
P01	Laptop	Electronics	55000
P02	Headphones	Electronics	3000
P03	Desk Chair	Furniture	12000
P04	Notebook	Stationery	150

CREATE TABLE — PRODUCTS

```
DROP TABLE IF EXISTS PRODUCTS;  
  
CREATE TABLE PRODUCTS (  
    PRODUCT_ID    VARCHAR(10)    NOT NULL,  
    PRODUCT_NAME  VARCHAR(100)   NOT NULL,  
    CATEGORY      VARCHAR(100)   NOT NULL,  
    PRICE         DECIMAL(10,2)  NOT NULL,  
    PRIMARY KEY (PRODUCT_ID)
```

INSERT Data — PRODUCTS

```
INSERT INTO PRODUCTS (PRODUCT_ID, PRODUCT_NAME, CATEGORY, PRICE) VALUES  
    ('P01', 'Laptop', 'Electronics', 55000.00),  
    ('P02', 'Headphones', 'Electronics', 3000.00),  
    ('P03', 'Desk Chair', 'Furniture', 12000.00),  
    ('P04', 'Notebook', 'Stationery', 150.00);
```

Table 3 of 3 — ORDERS

Preview of data:

ORDER_ID	CUSTOMER_ID	PRODUCT_ID	QUANTITY	ORDER_DATE	STATUS
1001	C01	P01	2	2024-01-05	Delivered
1002	C02	P02	1	2024-01-10	Pending
1003	C01	P03	5	2024-01-12	Delivered
1004	C03	P01	3	2024-01-15	Cancelled
1005	C02	P04	2	2024-02-01	Delivered
1006	C04	P02	1	2024-02-05	Pending
1007	C03	P03	4	2024-02-10	Delivered

CREATE TABLE — ORDERS

```
-- ORDERS depends on both CUSTOMERS and PRODUCTS - create it last
DROP TABLE IF EXISTS ORDERS;

CREATE TABLE ORDERS (
    ORDER_ID      INT          NOT NULL AUTO_INCREMENT,
    CUSTOMER_ID   VARCHAR(10)  NOT NULL,
    PRODUCT_ID    VARCHAR(10)  NOT NULL,
    QUANTITY      INT          NOT NULL,
    ORDER_DATE    DATE         NOT NULL,
    STATUS        VARCHAR(20)  NOT NULL,
    PRIMARY KEY (ORDER_ID),
    FOREIGN KEY (CUSTOMER_ID) REFERENCES CUSTOMERS (CUSTOMER_ID),
    FOREIGN KEY (PRODUCT_ID) REFERENCES PRODUCTS (PRODUCT_ID)
);
```

INSERT Data — ORDERS

```
-- STATUS values used: 'Delivered', 'Pending', 'Cancelled'
INSERT INTO ORDERS (CUSTOMER_ID, PRODUCT_ID, QUANTITY, ORDER_DATE, STATUS)
VALUES
    ('C01', 'P01', 2, '2024-01-05', 'Delivered'),
    ('C02', 'P02', 1, '2024-01-10', 'Pending'),
    ('C01', 'P03', 5, '2024-01-12', 'Delivered'),
    ('C03', 'P01', 3, '2024-01-15', 'Cancelled'),
    ('C02', 'P04', 2, '2024-02-01', 'Delivered'),
    ('C04', 'P02', 1, '2024-02-05', 'Pending'),
    ('C03', 'P03', 4, '2024-02-10', 'Delivered');
```

Verify the Data

```
-- Run these to confirm all rows loaded correctly
SELECT * FROM CUSTOMERS;
SELECT * FROM PRODUCTS;
SELECT * FROM ORDERS;

-- Quick check: count rows in each table
SELECT 'CUSTOMERS' AS tbl, COUNT(*) AS rows FROM CUSTOMERS
UNION ALL
SELECT 'PRODUCTS',          COUNT(*)          FROM PRODUCTS
UNION ALL
SELECT 'ORDERS',            COUNT(*)            FROM ORDERS;
```

Complete Script — Copy and Run at Once

The block below contains the full setup for both table sets in the correct order. You can copy this entire script and run it in one go in MySQL Workbench or the MySQL command line.

```
=====
SQL PRACTICE SETUP – Complete Script
Covers: DEPARTMENTS, EMPLOYEES, CUSTOMERS, PRODUCTS, ORDERS
=====

-- 1. Database
CREATE DATABASE IF NOT EXISTS sql_practice;
USE sql_practice;

-- 2. Drop tables in reverse dependency order
DROP TABLE IF EXISTS ORDERS;
DROP TABLE IF EXISTS EMPLOYEES;
DROP TABLE IF EXISTS DEPARTMENTS;
DROP TABLE IF EXISTS PRODUCTS;
DROP TABLE IF EXISTS CUSTOMERS;

-- 3. DEPARTMENTS
CREATE TABLE DEPARTMENTS (
    DEPT_ID      INT          NOT NULL AUTO_INCREMENT,
    DEPT_NAME    VARCHAR(50)  NOT NULL,
    LOCATION     VARCHAR(100) NOT NULL,
    PRIMARY KEY (DEPT_ID)
);
INSERT INTO DEPARTMENTS (DEPT_NAME, LOCATION) VALUES
    ('HR', 'Mumbai'), ('IT', 'Bangalore'), ('Finance', 'Delhi');

-- 4. EMPLOYEES (self-referencing; managers inserted first)
CREATE TABLE EMPLOYEES (
    EMP_ID      INT          NOT NULL,
    NAME        VARCHAR(100) NOT NULL,
    DEPARTMENT  VARCHAR(50)  NOT NULL,
    SALARY      DECIMAL(10,2) NOT NULL,
    MANAGER_ID  INT          NULL,
    PRIMARY KEY (EMP_ID),
    FOREIGN KEY (MANAGER_ID) REFERENCES EMPLOYEES(EMP_ID)
);
INSERT INTO EMPLOYEES VALUES
    (101, 'Alice', 'HR', 50000.00, NULL),
    (104, 'Diana', 'Finance', 90000.00, NULL);
INSERT INTO EMPLOYEES VALUES
    (102, 'Bob', 'IT', 80000.00, 101),
    (103, 'Charlie', 'IT', 75000.00, 101),
    (105, 'Eve', 'HR', 48000.00, 101),
    (106, 'Frank', 'Finance', 95000.00, 104),
    (107, 'Grace', 'IT', 82000.00, 101);

-- 5. CUSTOMERS
CREATE TABLE CUSTOMERS (
    CUSTOMER_ID  VARCHAR(10)  NOT NULL,
    CUSTOMER_NAME VARCHAR(100) NOT NULL,
    CITY         VARCHAR(100) NOT NULL,
    CREDIT_LIMIT DECIMAL(10,2) NOT NULL,
    PRIMARY KEY (CUSTOMER_ID)
```

```
);
INSERT INTO CUSTOMERS VALUES
  ('C01','Ravi Kumar', 'Delhi',    100000.00),
  ('C02','Priya Singh','Mumbai',   80000.00),
  ('C03','Amit Sharma','Bangalore', 60000.00),
  ('C04','Neha Joshi', 'Chennai',   50000.00);

-- 6. PRODUCTS
CREATE TABLE PRODUCTS (
  PRODUCT_ID   VARCHAR(10)   NOT NULL,
  PRODUCT_NAME VARCHAR(100)  NOT NULL,
  CATEGORY     VARCHAR(100)  NOT NULL,
  PRICE        DECIMAL(10,2) NOT NULL,
  PRIMARY KEY (PRODUCT_ID)
);
INSERT INTO PRODUCTS VALUES
  ('P01','Laptop',    'Electronics', 55000.00),
  ('P02','Headphones','Electronics',  3000.00),
  ('P03','Desk Chair','Furniture',    12000.00),
  ('P04','Notebook',  'Stationery',    150.00);

-- 7. ORDERS (depends on CUSTOMERS and PRODUCTS)
CREATE TABLE ORDERS (
  ORDER_ID      INT          NOT NULL AUTO_INCREMENT,
  CUSTOMER_ID   VARCHAR(10)  NOT NULL,
  PRODUCT_ID    VARCHAR(10)  NOT NULL,
  QUANTITY      INT          NOT NULL,
  ORDER_DATE    DATE         NOT NULL,
  STATUS        VARCHAR(20)  NOT NULL,
  PRIMARY KEY (ORDER_ID),
  FOREIGN KEY (CUSTOMER_ID) REFERENCES CUSTOMERS (CUSTOMER_ID),
  FOREIGN KEY (PRODUCT_ID) REFERENCES PRODUCTS (PRODUCT_ID)
);
INSERT INTO ORDERS (CUSTOMER_ID,PRODUCT_ID,QUANTITY,ORDER_DATE,STATUS) VALUES
  ('C01','P01',2,'2024-01-05','Delivered'),
  ('C02','P02',1,'2024-01-10','Pending'),
  ('C01','P03',5,'2024-01-12','Delivered'),
  ('C03','P01',3,'2024-01-15','Cancelled'),
  ('C02','P04',2,'2024-02-01','Delivered'),
  ('C04','P02',1,'2024-02-05','Pending'),
  ('C03','P03',4,'2024-02-10','Delivered');

-- Verification
SELECT 'DEPARTMENTS' AS tbl, COUNT(*) AS rows FROM DEPARTMENTS
UNION ALL SELECT 'EMPLOYEES', COUNT(*) FROM EMPLOYEES
UNION ALL SELECT 'CUSTOMERS',  COUNT(*) FROM CUSTOMERS
UNION ALL SELECT 'PRODUCTS',   COUNT(*) FROM PRODUCTS
UNION ALL SELECT 'ORDERS',     COUNT(*) FROM ORDERS;

-- Expected row counts: DEPARTMENTS=3, EMPLOYEES=7,
-- CUSTOMERS=4, PRODUCTS=4, ORDERS=7
```

Quick Column Reference

Use this table as a quick reference while writing queries.

Table Name	Column Name	Data Type	Notes
DEPARTMENTS	DEPT_ID	INT (AUTO)	Primary Key, auto-generated
DEPARTMENTS	DEPT_NAME	VARCHAR (50)	HR / IT / Finance
DEPARTMENTS	LOCATION	VARCHAR (100)	City of department
EMPLOYEES	EMP_ID	INT	Primary Key
EMPLOYEES	NAME	VARCHAR (100)	Employee full name
EMPLOYEES	DEPARTMENT	VARCHAR (50)	Matches DEPT_NAME in DEPARTMENTS
EMPLOYEES	SALARY	DECIMAL (10,2)	Monthly salary in INR
EMPLOYEES	MANAGER_ID	INT (NULL)	Self-ref FK to EMP_ID; NULL = top manager
CUSTOMERS	CUSTOMER_ID	VARCHAR (10)	Primary Key (C01, C02 ...)
CUSTOMERS	CUSTOMER_NAME	VARCHAR (100)	Full name
CUSTOMERS	CITY	VARCHAR (100)	City of the customer
CUSTOMERS	CREDIT_LIMIT	DECIMAL (10,2)	Maximum credit in INR
PRODUCTS	PRODUCT_ID	VARCHAR (10)	Primary Key (P01, P02 ...)
PRODUCTS	PRODUCT_NAME	VARCHAR (100)	Name of product
PRODUCTS	CATEGORY	VARCHAR (100)	Electronics / Furniture / Stationery
PRODUCTS	PRICE	DECIMAL (10,2)	Unit price in INR
ORDERS	ORDER_ID	INT (AUTO)	Primary Key, auto-generated
ORDERS	CUSTOMER_ID	VARCHAR (10)	FK to CUSTOMERS.CUSTOMER_ID
ORDERS	PRODUCT_ID	VARCHAR (10)	FK to PRODUCTS.PRODUCT_ID
ORDERS	QUANTITY	INT	Number of units ordered
ORDERS	ORDER_DATE	DATE	Format: YYYY-MM-DD
ORDERS	STATUS	VARCHAR (20)	Delivered / Pending / Cancelled

Tip: All DECIMAL(10,2) columns store values with 2 decimal places. When you run SELECT queries the output will show values like 50000.00. This is expected and correct.